Uganda GovHub API
Government Systems Prototype Showcase

ARCHITECTURE SUBMISSION

System Architecture for Governed MDA API Interoperability

A structured description of the prototype's user access model, frontend and backend services, data stores, sandbox integrations, security controls, hosting posture, and data-flow boundaries.

THEMATIC AREA	SUBMISSION SCOPE	PREPARED FOR
Area 10: Interoperability and Data Exchange.	Working local prototype using deterministic mock records only.	MoICT&NG Government Systems Prototype Showcase.

This document follows the system architecture guidance by showing the main components, communication paths, storage and security controls, user access routes, APIs and integrations, hosting posture, and practical readiness considerations.

What the system is designed to do

Uganda GovHub API is a developer portal, sandbox, and governance console for Ministries, Departments, and Agencies. It demonstrates how government APIs can be discovered, documented, requested, approved, tested, and audited without exposing live citizen records.

5
Seeded showcase APIs for identity, tax, business registration, driving permit, and composite eligibility.

4
Operational demo roles: developer, API owner, compliance reviewer, and platform administrator.

1
Unified workflow for API discovery, access approval, sandbox execution, and audit review.

Current implementation

The prototype is a local full-stack web application. The frontend is built with React, TypeScript, Vite, React Router, and GovHub UI components. The backend is Node.js, Express, TypeScript, OpenAPI YAML files, and SQLite through `better-sqlite3`.

`/` API catalog and API detail flow	`/docs` Visible OpenAPI documentation index
`/dashboard` Approvals, access matrix, analytics, audit logs	`/catalog/add` OpenAPI validation and API registration

Government deployment intent

The architecture separates public discovery from operational access. It places policy metadata next to API contracts, requires account and API-level approval, scopes sandbox credentials, and records audit events with correlation IDs for traceability.

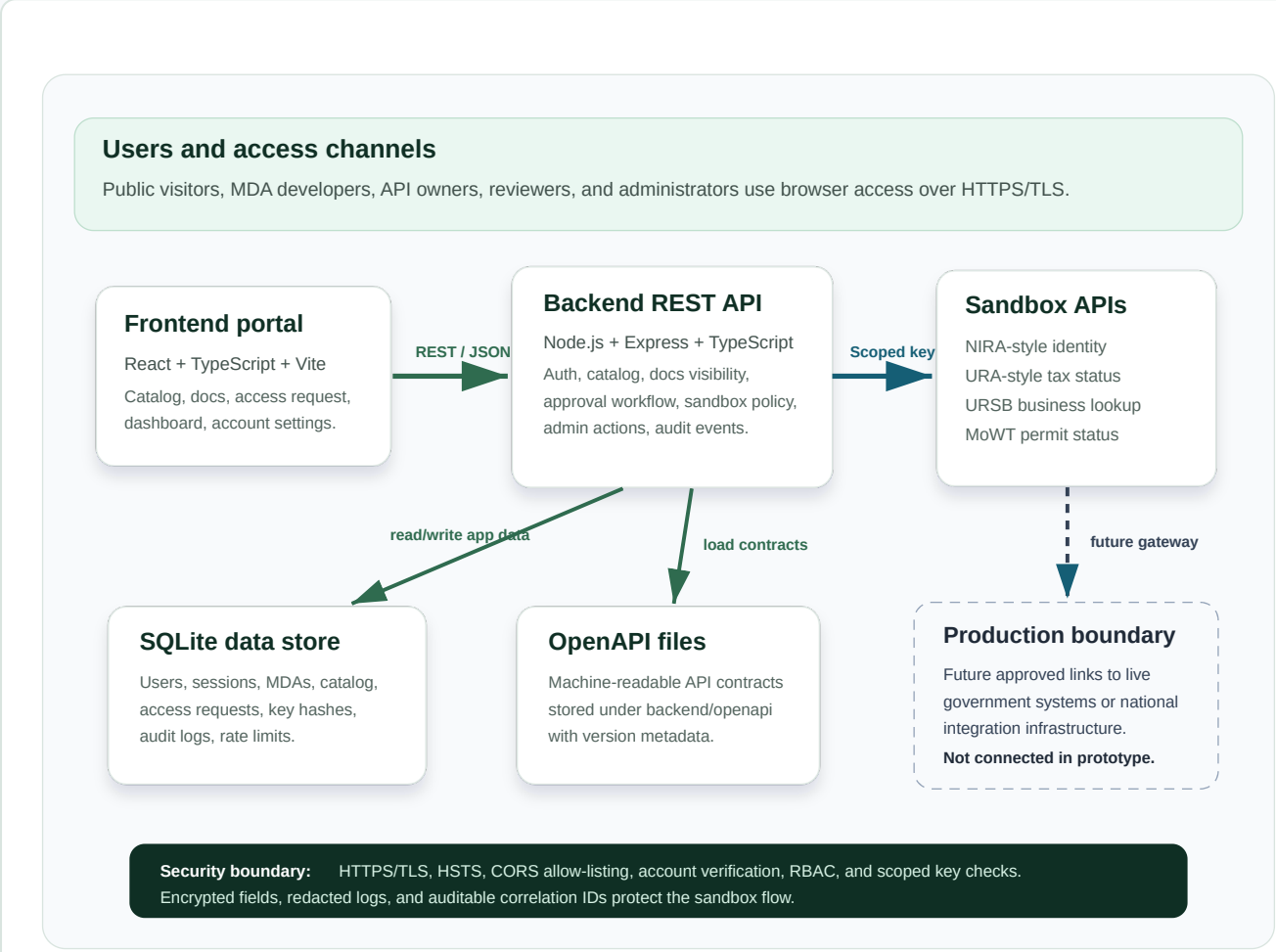
- No live connection to NIRA, URA, URSB, MoWT, NITA-U, or other production systems in the prototype.
- Mock records are deterministic so a ministry presenter can repeat the demo safely.
- The design can be upgraded to production infrastructure without changing the core governance workflow.

Coverage of the architecture guidance

- ✓ Main system components and how they interact.
- ✓ Database, OpenAPI storage, and security posture.
- ✓ Hosting model, scalability path, and single-point-of-failure mitigations.
- ✓ User access through browser-based workflows.
- ✓ APIs exposed, sandbox services consumed, and external-system boundaries.
- ✓ Data flow, audit trail, encryption, authentication, and access control.

System components and data exchange paths

The diagram shows how users reach the React portal, how the portal calls the Express backend, where data and specifications are stored, and where sandbox API traffic is handled. All external government systems are represented as a production boundary, not as live integrations in this prototype.



Architecture components requested by the guidance

The prototype presents the main system parts in a way a technical evaluation panel can review for fitness, interoperability, security, maintainability, and transferability.

	IMPLEMENTED DESIGN	WHAT IT SHOWS FOR EVALUATION
Frontend / user interface	React, TypeScript, Vite, React Router, dashboard UI components, API docs pages, account pages, and catalog workflows.	Users access through a browser. The interface supports public discovery, verified account operations, access requests, approvals, audit review, and API onboarding.
Backend / application layer	Node.js, Express, TypeScript routes, middleware, policy helpers, OpenAPI validation, docs visibility, access control, audit service, and sandbox handlers.	Business logic is centralized behind REST endpoints. Requests are processed by auth, role, API visibility, sandbox scope, rate-limit, and audit controls.
Database layer	SQLite through `better-sqlite3` for prototype persistence; schema includes users, sessions, MDAs, APIs, API versions, access requests, audit logs, and rate limits.	Data categories and relationships are explicit. A production build can move the same logical model to managed PostgreSQL or another government-approved database service.
Authentication and access	Signup/login, account review states, seeded demo accounts, role-based access, MFA support, bearer sessions, scoped sandbox API keys, expiry, revocation, and docs visibility rules.	Public transparency is separated from operational trust. Developers, API owners, reviewers, and administrators have distinct permissions.
APIs and integrations	Catalog and docs APIs plus sandbox endpoints for identity, tax, business registration, driving permit, and composite eligibility scenarios.	The system exposes documented APIs and consumes only internal mock handlers in the prototype. Live NIRA, URA, URSB, MoWT, and NITA-U systems are not called.
Hosting and infrastructure	Local showcase deployment: Vite frontend dev server, Express backend on `127.0.0.1:4000`, local SQLite database, local OpenAPI files.	Current hosting is sufficient for demonstration. Production should use TLS termination, load-balanced app services, managed database, backup, monitoring, and secret management.
Security controls	TLS/HSTS hooks, CORS allow-listing, JSON body limits, password hashing, encrypted sensitive fields, API key hashing, rate limits, correlation IDs, redacted logs, and structured denial responses.	Security is treated as an architectural property across user access, data storage, transport, API execution, and audit review.
Data flow	User submits requests through the portal; backend validates auth and role; SQLite stores approvals and audit events; sandbox handlers return mock responses with correlation IDs.	The data boundary is visible: mock inputs enter the sandbox, access decisions are logged, and no live personal or production government records leave external systems.

Maintainability note: The stack uses common transferable technologies: React, TypeScript, Node.js, Express, OpenAPI, and SQL persistence. Production hardening would focus on infrastructure, secrets, observability, and database migration rather than replacing the core application workflow.

How data moves through the prototype

The core flow starts with API discovery and ends with an audited sandbox transaction. Access is explicit, credentials are scoped, and every allowed or denied sandbox call can be traced by correlation ID.

1. **Discover**
   User browses the catalog and reads governance metadata and permitted docs.
2. **Sign in**
   Verified account state and role determine which operations are visible.
3. **Request**
   Developer states purpose, legal basis, requested fields, volume, and environment.
4. **Approve**
   Owner or administrator reviews and issues a scoped sandbox key.
5. **Execute**
   Sandbox middleware checks key hash, scope, status, expiry, and rate limit.
6. **Audit**
   Allowed and denied calls are logged with correlation IDs and redacted values.

Stored data categories

DATA	STORAGE AND PROTECTION
Catalog and governance metadata	SQLite rows for APIs, MDAs, lifecycle state, sensitivity, purpose limitation, retention, and compliance fields.
OpenAPI contracts	YAML files under `backend/openapi` with version records and visibility checks before disclosure.
User and session records	Password hashes, session records, account review state, requested/approved role, and approved MDA.
Access requests and keys	Business purpose, legal basis, requested fields, status, expiry, revocation state, and API key hash only.
Audit logs	Event type, MDA/API context, actor where available, correlation ID, timestamps, and redacted metadata.

Implemented security controls

Transport
TLS certificate/key support and HSTS when TLS or trusted termination is enabled.

Boundary
CORS origin allow-list and JSON request body size limits.

Identity
Verified account states, role-based access, MFA support, and seeded demo identities.

Storage
AES-256-GCM field encryption support for sensitive values and hash-only API key storage.

Execution
Scoped sandbox keys, expiry, revocation, endpoint authorization, and rate limits.

Traceability
Server-generated correlation IDs, audit events, and sensitive-value redaction.

APIs represented in the prototype

The seeded API catalog demonstrates how several MDAs could publish governed, documented services through one discovery and sandbox experience. The implementation uses mock handlers only; it does not call production government systems.

	API PRODUCT	PRIMARY INTEROPERABILITY VALUE	CLASSIFICATION
NIRA	Identity Verification	Verify identity match and return the minimum fields needed for a lawful service workflow.	Restricted
URA	Tax Compliance Status	Check tax-clearance style status for eligibility, procurement, and compliance workflows.	Official
URSB	Business Registration Lookup	Validate business registration facts, current status, and registry details.	Public
MoWT	Driving Permit Verification	Confirm permit validity, class authorization, expiry, and suspension state.	Official
MoICT	Service Uganda Composite Eligibility	Show multi-agency orchestration using approved sandbox scopes and correlation IDs.	Restricted

CURRENT PROTOTYPE

Internal sandbox simulation

Requests terminate inside the Express backend. The middleware validates key status and scope, then route handlers return deterministic mock responses and structured errors. This keeps the showcase safe and repeatable.

PRODUCTION PATHWAY

Approved integration gateway

A production deployment would connect through approved national integration infrastructure or MDA gateways, with contract testing, adapter services, message signing, monitoring, and data-sharing agreements.

GOVERNANCE BOUNDARY

Owner accountability preserved

Each API keeps an owning MDA, technical owner, approval authority, statutory basis, purpose limitation, retention class, and audit requirement. Access is not global simply because the API is discoverable.

External-system statement: NIRA, URA, URSB, MoWT, NITA-U, and other production government systems are not integrated in this prototype. They are represented through realistic mock contracts and sandbox handlers to demonstrate interoperability design without exposing live government data.

Path from showcase prototype to government deployment

The local prototype is intentionally small, but the architecture identifies where production hardening would be applied: runtime isolation, database migration, secret management, monitoring, backup, and approved integration channels.

Current hosting model

- Frontend runs as a Vite React application for local demonstration.
- Backend runs as an Express service, defaulting to `http://127.0.0.1:4000`.
- SQLite database and OpenAPI files are local to the backend workspace.
- Dependencies are installed locally; no live external API access is required for the demo flow.

Production hardening path

- Serve React static assets behind a government-approved web host or reverse proxy.
- Run backend services in load-balanced containers or managed application services.
- Migrate SQLite to managed PostgreSQL or an approved relational database platform.
- Move OpenAPI files to versioned object storage or database-backed spec storage.
- Use managed secrets, KMS-backed encryption keys, centralized logs, metrics, and backups.

SCALABILITY

Scale application and storage independently

The REST API is stateless apart from database and file storage dependencies. In production, multiple backend instances can serve traffic behind a load balancer once sessions, database, and files are centralized.

RESILIENCE

Remove local single points of failure

The prototype's local SQLite file and local OpenAPI directory are intentional demo choices. Production should use replicated storage, backups, disaster recovery, health checks, and deployment rollback.

SECURITY OPERATIONS

Strengthen credentials and audit assurance

Move sandbox access from shared API keys toward asymmetric request signing, key rotation, nonce replay protection, and tamper-evident audit anchoring where required.

	WHERE IT IS ADDRESSED IN THIS SUBMISSION
Main components	Sections 02 and 03 show the frontend, backend, database, OpenAPI storage, sandbox services, and production boundary.
Communication and protocols	Sections 02 and 04 show browser access, REST/JSON API calls, scoped sandbox key checks, HTTPS/TLS posture, and correlation IDs.
Data storage and security	Sections 03 and 04 describe SQLite data categories, OpenAPI files, encrypted fields, key hashing, rate limits, redaction, and audit logging.
User access	Sections 01, 03, and 04 describe public visitors, verified developers, API owners, reviewers, administrators, and account review states.
External integrations	Section 05 identifies NIRA-style, URA-style, URSB-style, MoWT-style, and composite sandbox APIs, and clearly states that production systems are not called.
Hosting and infrastructure	Section 06 distinguishes the local showcase environment from a production hardening path for government deployment.